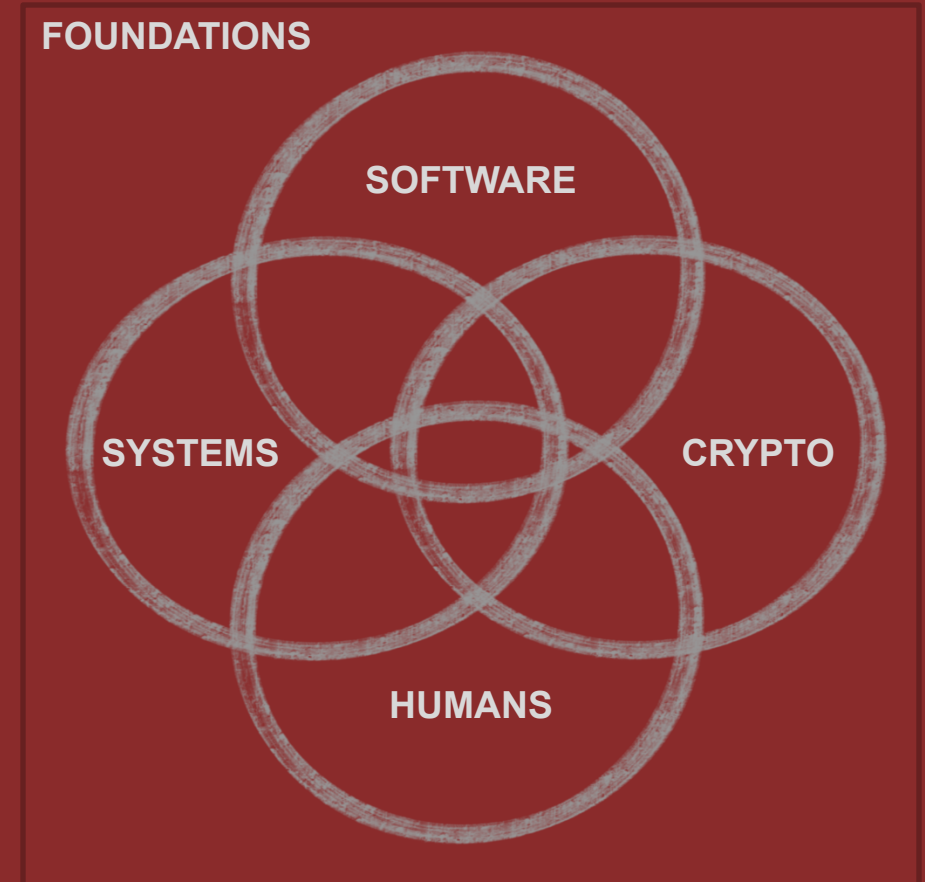


Διάλεξη #17 - Hash Functions

Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών

Εισαγωγή στην Ασφάλεια

Θανάσης Αυγερινός



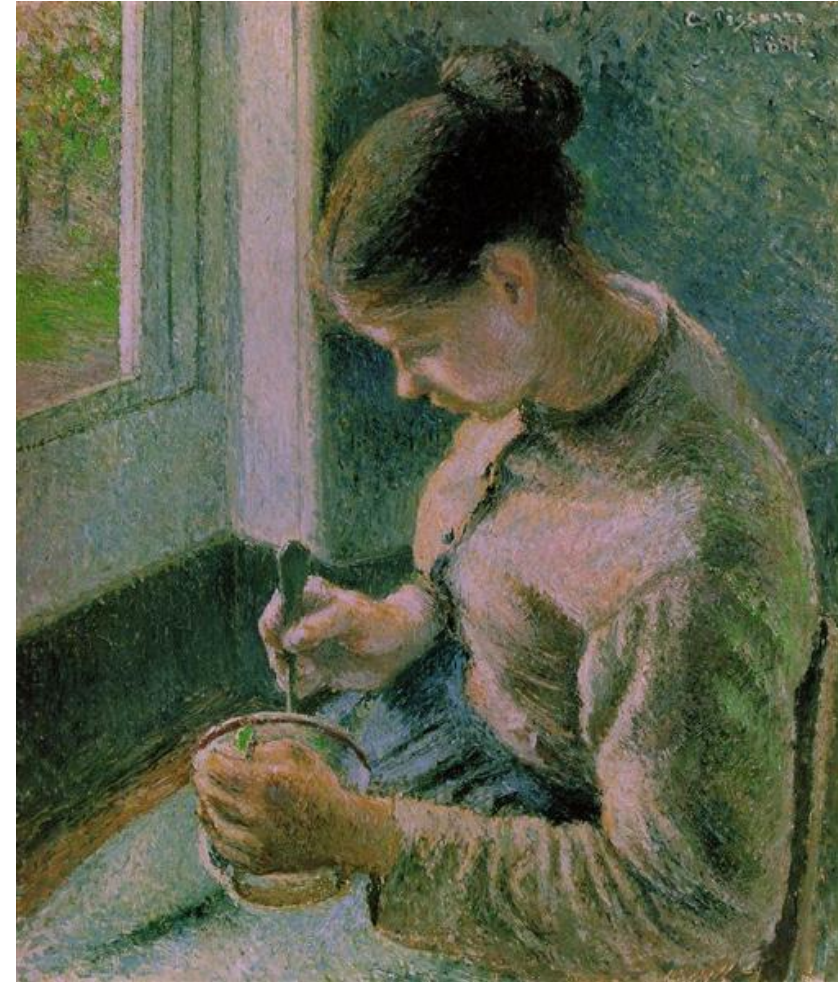
Huge thank you to [David Brumley](#) from Carnegie Mellon University for the guidance and content input while developing this class (lots of slides from Dan Boneh @ Stanford and some from Adrian Perrig)

Ανακοινώσεις / Διευκρινίσεις

1. Αναπλήρωση την Παρασκευή - 9-11πμ @ Αίθουσα A2
2. Title: Trustworthy Operating Systems
 - a. Speaker: Vasileios (Vasilis) Kemerlis
 - b. Date/Time: May 7th / 15:00-16:00

Την προηγούμενη φορά

- Message Integrity
 - Message Authentication Codes (MACs)
 - CBC-MAC, NMAC, CMAC
- Introduction to Hashing



Σήμερα

- Hashes Intro
- Hash Constructions
- HMAC
- Hash Tricks/Datastructures





Hashes and Resistance

Cryptographic Hash Functions

A Cryptographic Hash Function (CHF) is an algorithm that maps an arbitrary binary string to a string of n bits. $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$

- Message space much larger than output space

$$H: M \rightarrow T, |M| \gg |T|$$

- Given the output, we want the input to remain secret and also make it hard for other inputs to get the same output (collision).
- Applications: everywhere (from storing passwords,

Hash Function Properties

Let $H: M \rightarrow T$, $|M| \gg |T|$

- **Pre-image resistance.** H is pre-image resistant if given a hash value h , it should be difficult to find any message m such that $H(m) = h$. In other words, $P[H(\text{random } m) = h] = 1/|T|$.
- **Second pre-image resistance (weak collision resistance).** H is second-preimage resistant if given a message m_1 , it should be difficult to find a different m_2 such that $H(m_1) = H(m_2)$.
- **(Strong) Collision resistance.** H is collision resistant if it is difficult to find any two different messages m_1 and m_2 such that $H(m_1) = H(m_2)$.

Collision Resistance =>
Second-preimage Resistance

Second-preimage Resistance => Preimage Resistance?

*only true under certain conditions ($|M| \gg |T|$)

Collision Resistance Definition

Let $H: M \rightarrow T$ be a hash function $(|M| \gg |T|)$

A **collision** for H is a pair $m_0, m_1 \in M$ such that:

$$H(m_0) = H(m_1) \quad \text{and} \quad m_0 \neq m_1$$

A function H is **collision resistant** if for all (explicit) “eff” algs. A :

$$\text{Adv}_{\text{CR}}[A, H] = \Pr[A \text{ outputs collision for } H]$$

is “neg”.

Example: SHA-256 (outputs 256 bits)

Generic attack on C.R. functions

Let $H: M \rightarrow \{0,1\}^n$ be a hash function ($|M| \gg 2^n$)

Generic alg. to find a collision **in time** $O(2^{n/2})$ hashes

Algorithm:

1. Choose $2^{n/2}$ random messages in M : $m_1, \dots, m_{2^{n/2}}$ (distinct w.h.p.)
2. For $i = 1, \dots, 2^{n/2}$ compute $t_i = H(m_i) \in \{0,1\}^n$
3. Look for a collision ($t_i = t_j$). If not found, got back to step 1.

How well will this work?

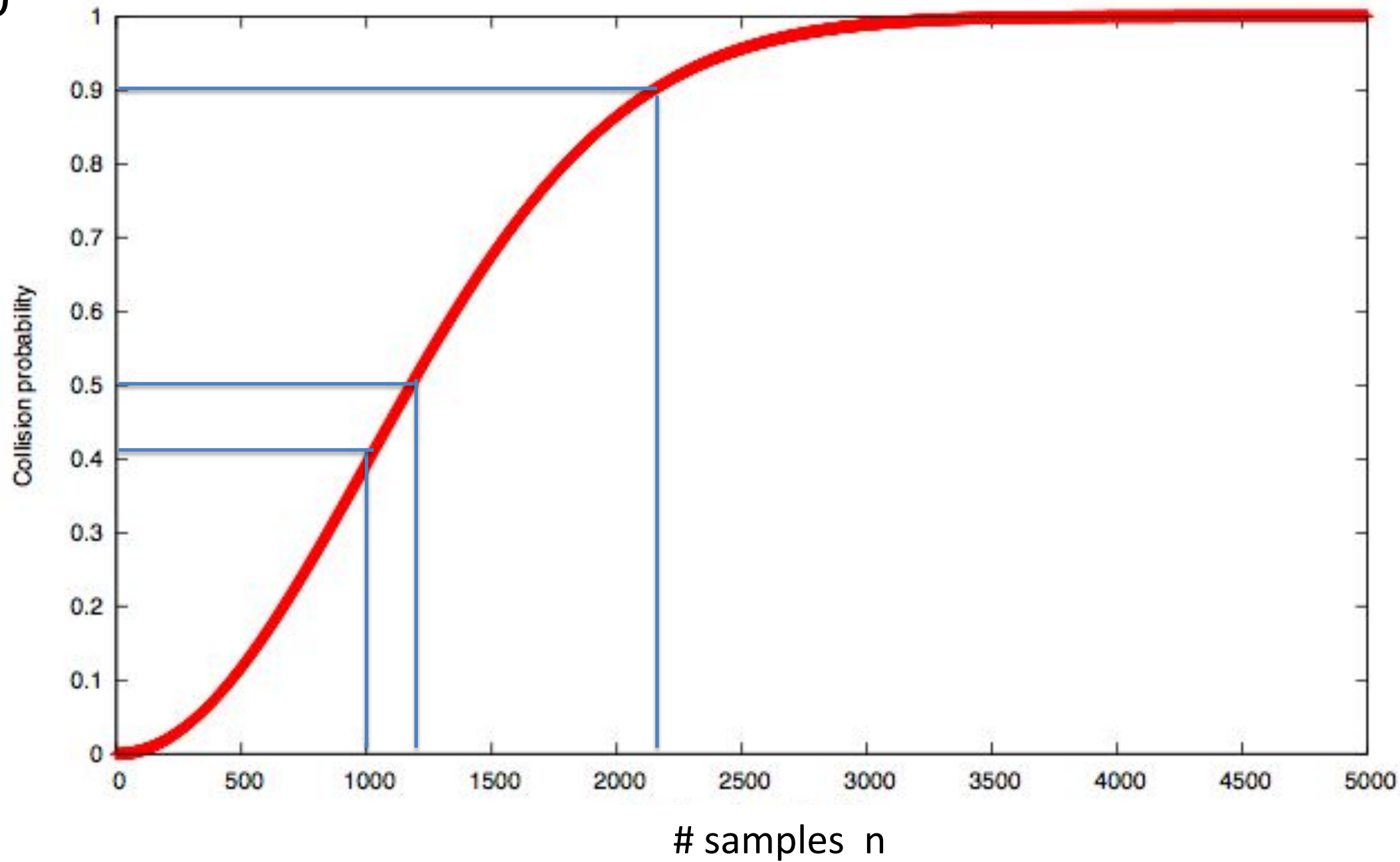
The birthday paradox

Let $r_1, \dots, r_n \in \{1, \dots, B\}$ be indep. identically distributed integers.

Thm: when $n = 1.2 \times B^{1/2}$ then $\Pr[\exists i \neq j: r_i = r_j] \geq \frac{1}{2}$

Proof: (for uniform indep. $r_1, \dots, r_n$)

$B=10^6$



Generic attack

$H: M \rightarrow \{0,1\}^n$. Collision finding algorithm:

1. Choose $2^{n/2}$ random elements in M : $m_1, \dots, m_{2^{n/2}}$
2. For $i = 1, \dots, 2^{n/2}$ compute $t_i = H(m_i) \in \{0,1\}^n$
3. Look for a collision ($t_i = t_j$). If not found, got back to step 1.

Expected number of iterations ≈ 2

Running time: **$O(2^{n/2})$** (space $O(2^{n/2})$)

Sample C.R. hash functions: Crypto++ 5.6.0 [Wei Dai]

AMD Opteron, 2.2 GHz (Linux)

	<u>function</u>	<u>digest size (bits)</u>	<u>Speed (MB/sec)</u>	<u>generic attack time</u>
NIST standards	SHA-1	160	153	2^{80}
	SHA-256	256	111	2^{128}
	SHA-512	512	99	2^{256}
	Whirlpool	512	57	2^{256}

* best known collision finder for SHA-1 requires 2^{51} hash evaluations

Collision attack: **same** hashes



Good doc



Sha-1



3713..42



Bad doc



Sha-1



3713..42

<https://marc-stevens.nl/research/shattered.io/>

SHA-256 Support #12490

Unanswered

dezren39 asked this question in **General**



dezren39 on Mar 7, 2022

Hi! A year ago it was mentioned that SHA-256 support was on the roadmap:

<https://github.community/t/support-for-sha-256-hashes/157493>

I was hoping to get an update on this and a timeline. Could we get a beta this year?



36



4



3

Quantum Collision Finder

	Classical algorithms	Quantum algorithms
Block cipher $E: K \times X \rightarrow X$ exhaustive search	$O(K)$	$O(K ^{1/2})$
Hash function $H: M \rightarrow T$ collision finder	$O(T ^{1/2})$	$O(T ^{1/3})$



Hash Constructions: Merkle-Damgård

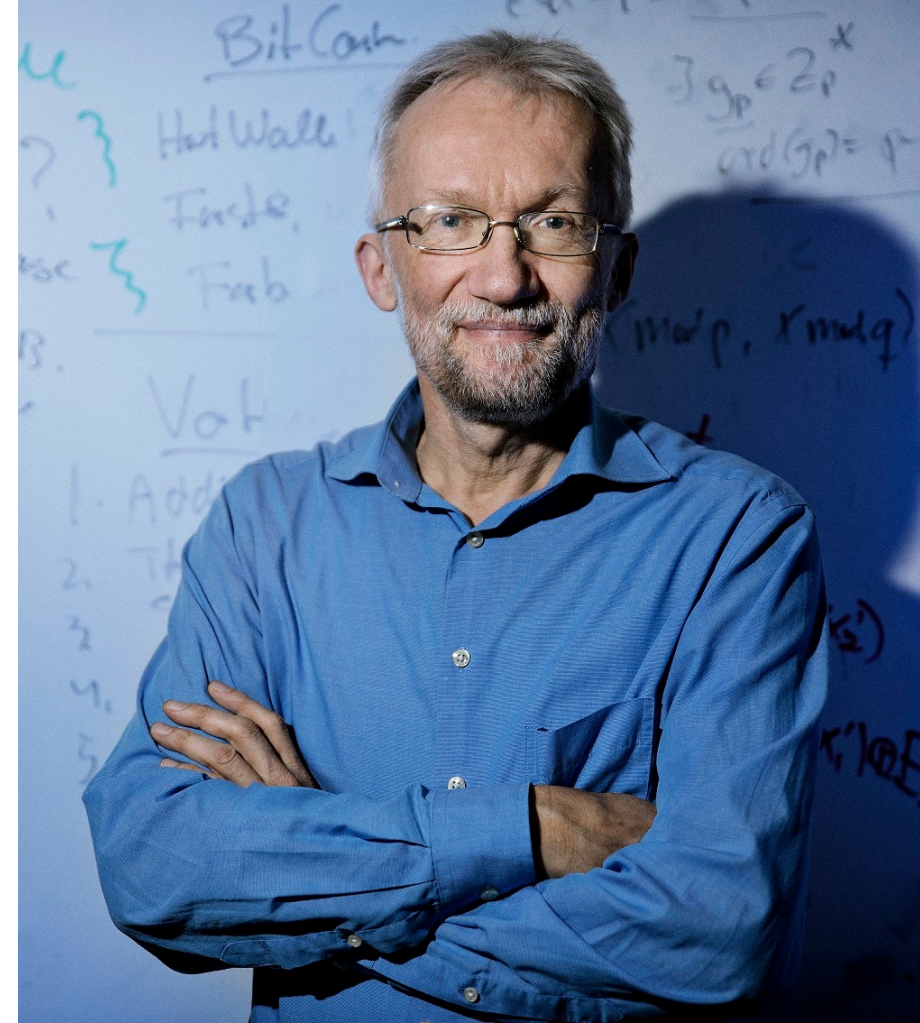
Collision resistance

Let $H: M \rightarrow T$ be a hash function ($|M| \gg |T|$)

A **collision** for H is a pair $m_0, m_1 \in M$ such that:
 $H(m_0) = H(m_1)$ and $m_0 \neq m_1$

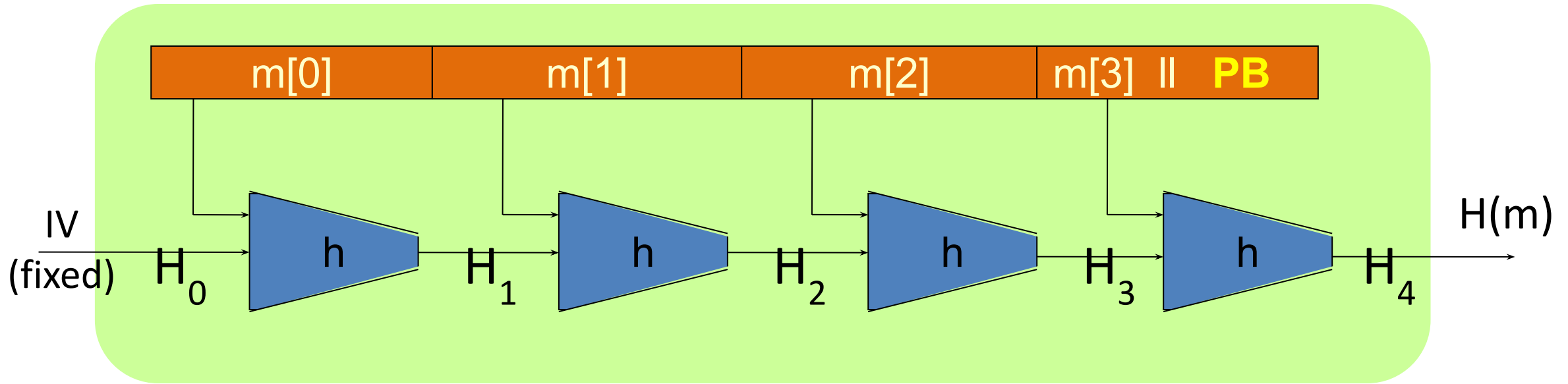
Goal: collision resistant (C.R.) hash functions

Step 1: given C.R. function for **short** messages,
construct C.R. function for **long** messages



Ivan Damgård co-inventor of the
Merkle-Damgård construction (1989)

The Merkle-Damgård iterated construction



Given $h: T \times X \rightarrow T$ (compression function)

we obtain $H: X^{\leq L} \rightarrow T$. H_i - chaining variables

PB: padding block



If no space for PB
add another block

MD collision resistance

Thm: if h is collision resistant then so is H .

Proof: collision on $H \Rightarrow$ collision on h

Suppose $H(M) = H(M')$. We build collision for h .

$$IV = H_0, H_1, \dots, H_t, H_{t+1} = H(M)$$

$$IV = H'_0, H'_1, \dots, H'_r, H'_{r+1} = H(M')$$

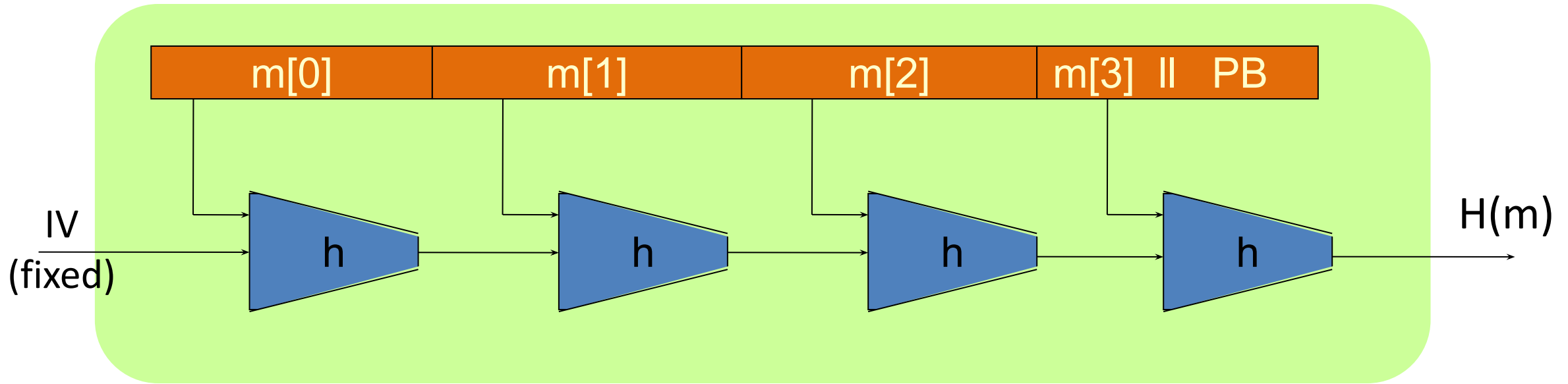
$$h(H_t, M_t \parallel PB) = H_{t+1} = H'_{r+1} = h(H'_r, M'_r \parallel PB')$$

Suppose $H_t = H'_r$ and $M_t = M'_r$ and $PB = PB'$

Then: $h(H_{t-1}, M_{t-1}) = H_t = H'_t = h(H'_{t-1}, M'_{t-1})$

$\Rightarrow$ To construct C.R. function,
suffices to construct compression function

The Merkle-Damgård iterated construction



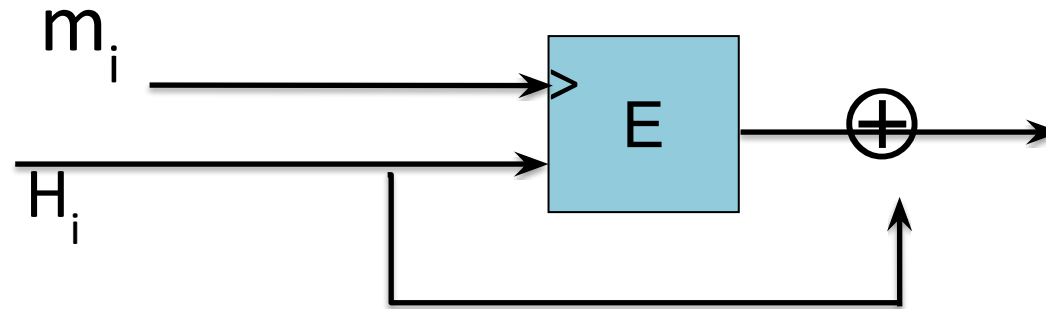
Thm: h collision resistant $\Rightarrow H$ collision resistant

Goal: construct compression function $h: T \times X \rightarrow T$

Compr. func. from a block cipher

$E: K \times \{0,1\}^n \rightarrow \{0,1\}^n$ a block cipher.

The **Davies-Meyer** compression function: $h(H, m) = E(m, H) \oplus H$



Thm: Suppose E is an ideal cipher (collection of $|K|$ random perms.). Finding a collision $h(H, m) = h(H', m')$ takes $O(2^{n/2})$ evaluations of (E, D) .

Best possible !!

[Winternitz 1984]

Suppose we define $h(H, m) = E(m, H)$

Then the resulting $h(.,.)$ is not collision resistant:

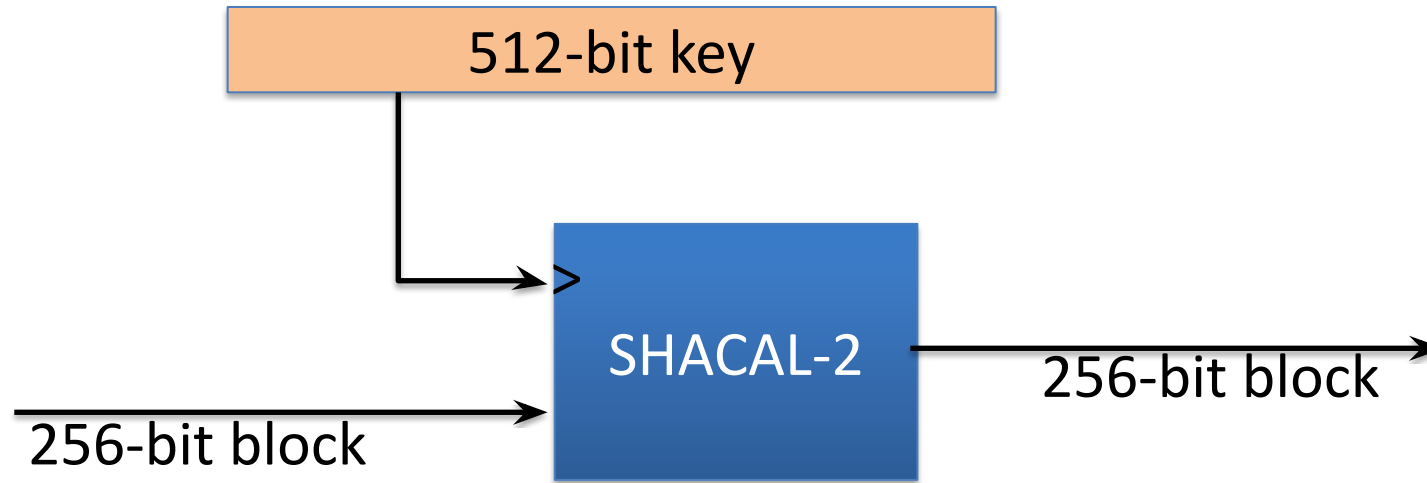
to build a collision (H, m) and (H', m')

choose random (H, m, m') and construct H' as follows:

- ☐ $H' = D(m', E(m, H))$
- ☐ $H' = E(m', D(m, H))$
- ☐ $H' = E(m', E(m, H))$
- ☐ $H' = D(m', D(m, H))$

Case study: SHA-256

- Merkle-Damgard function
- Davies-Meyer compression function
- Block cipher: SHACAL-2





Hashes and Passwords - An Interlude

Hash Functions are typically ***Fast***
> 10^6 / s on modern hardware

Some Hash Functions Are Slow

[PBKDF2](#) is ~5 orders of magnitude (100,000x) slower than a standard hash function (e.g., MD5). It is also the main recommendation for storing passwords (RFC 8018 / 2017).

1. Why is a hash function used for storing passwords?
2. Is slowness an advantage or disadvantage?

Careful with storing passwords

<https://hashcat.net/hashcat/>

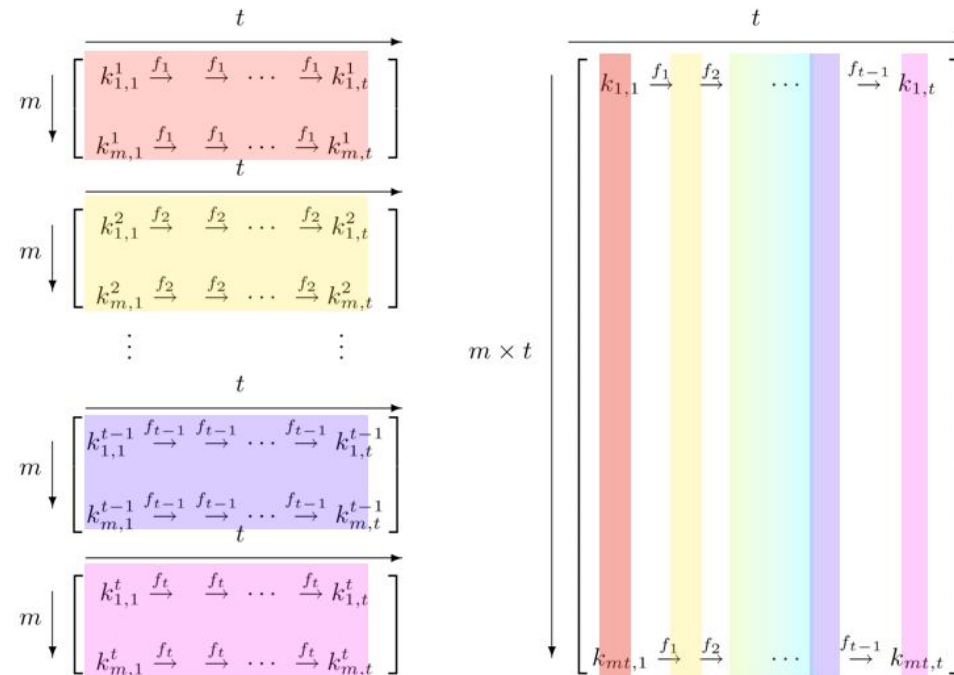


COMPONENT	PERCENTILE RANK	# COMPATIBLE PUBLIC RESULTS	H/S (AVERAGE)
NVIDIA GeForce RTX 4090	96th	28	152416197859 +/- 6710203881
MSI NVIDIA GeForce RTX 4090	96th	5	151733333333 +/- 6137634362
Zotac NVIDIA GeForce RTX 2080 Ti	90th	3	130494289583 +/- 210426329
NVIDIA GeForce RTX 4080	87th	14	94912651871 +/- 2409779704
Gigabyte NVIDIA GeForce RTX 3070	82nd	4	80013741667 +/- 228221564
NVIDIA GeForce RTX 3090 Ti	81st	4	75184916667 +/- 3931783726
Gigabyte NVIDIA GeForce RTX 4070 Ti	81st	4	74021816667 +/- 1638545931
Mid-Tier	75th		< 70991033333
NVIDIA GeForce RTX 3090	75th	39	70867552587 +/- 3204264127
AMD Radeon RX 7900 XTX	73rd	4	69163372857 +/- 1246558898
NVIDIA GeForce RTX 3080 Ti	72nd	14	67781101282 +/- 413951882
AMD Radeon RX 7900 XT	67th	5	61566224762 +/- 753630529
NVIDIA GeForce RTX 3080	66th	19	60284979323 +/- 871234740
AMD Radeon RX 6900 XT	63rd	9	58596096296 +/- 1383732339
AMD Radeon RX 6800 XT	56th	6	52565837302 +/- 1856012111
NVIDIA GeForce RTX 2080 SUPER	54th	3	43272383333
NVIDIA GeForce RTX 3070 Ti	52nd	12	42679897024 +/- 218910662

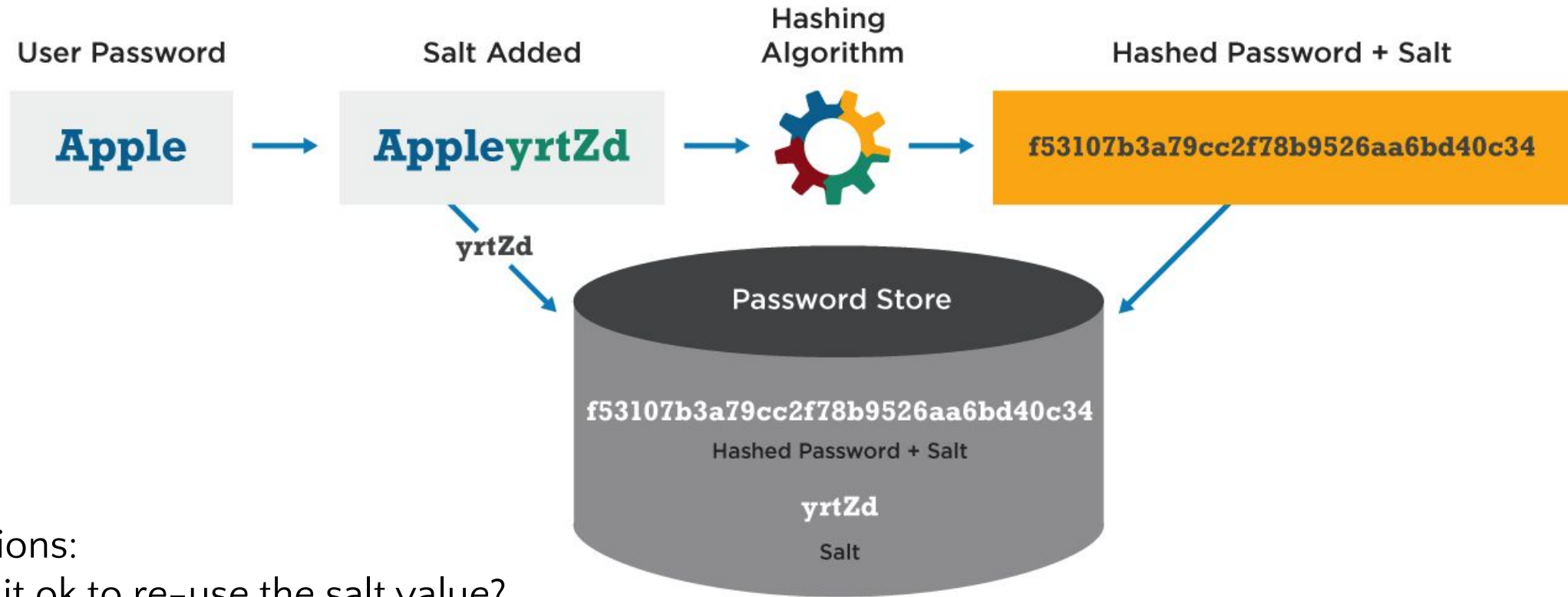
We just recovered the MD5 hash of a password:
d50ba4dd3fe42e17e9faa9ec29f89708 . Can we get the
original password?

Rainbow Tables

A [rainbow table](#) is a precompute table for caching the outputs of a hash function. Typically used for cracking password hashes. A common defense against this attack is to compute the hashes using a [key derivation function](#) that adds a "[salt](#)" to each password before hashing it, with different passwords receiving different salts, which are stored in plain text along with the hash.



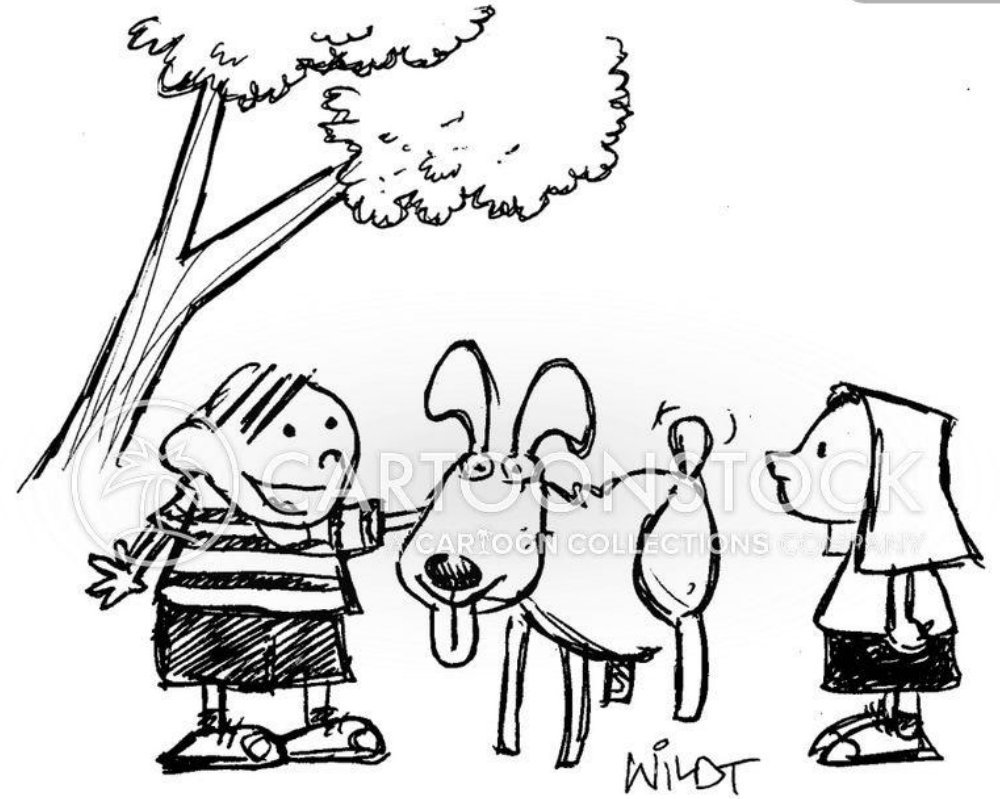
Password Hash Salting



Questions:

1. Is it ok to re-use the salt value?
2. Is it ok to use a 3-bit salt value?

No!



"Meet my very first dog.
I've named him 'Security Question.'"

Passwords may be Predictable

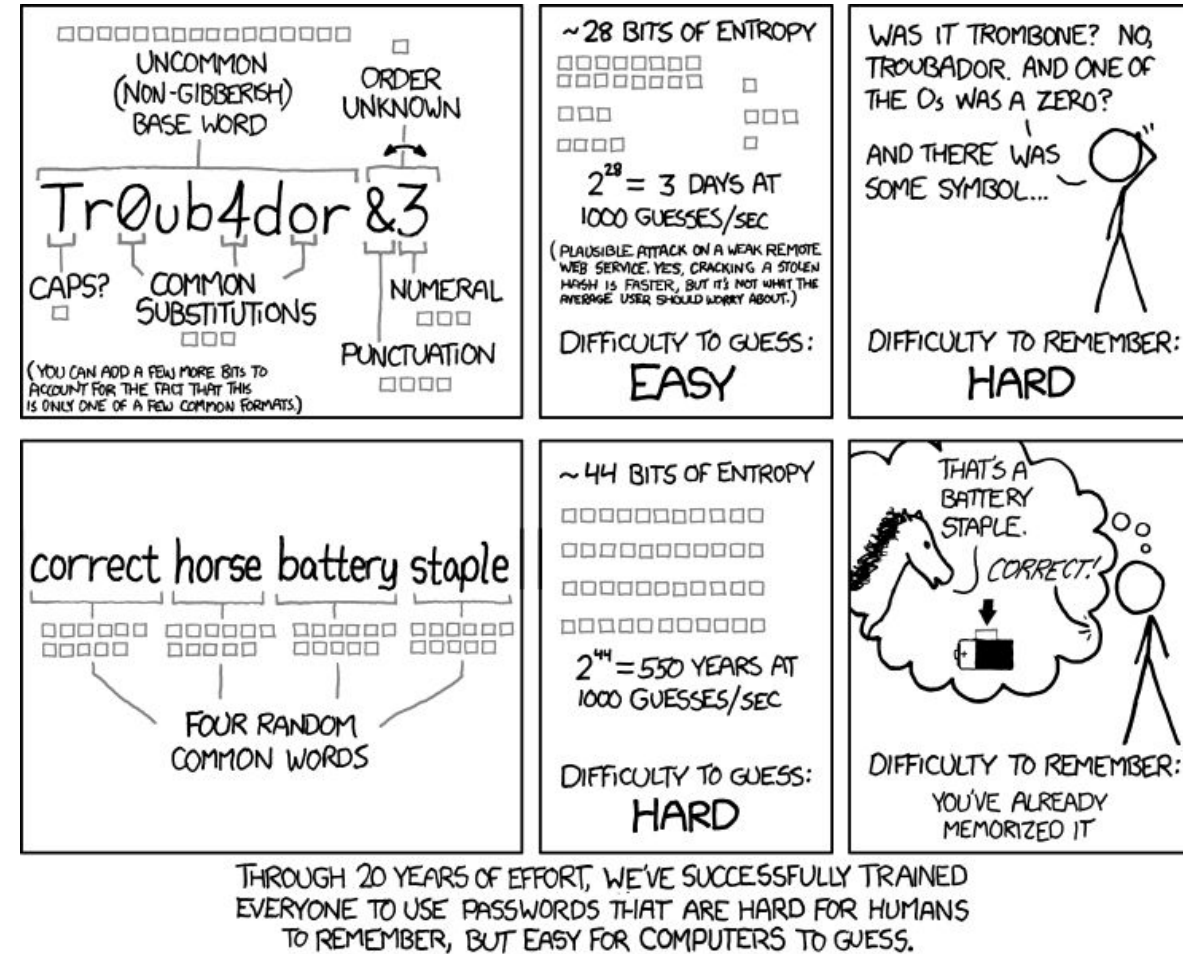
- Many users still choose obvious passwords: 123456 remains the most common [globally](#).
- Attackers use leaked password databases for dictionary attacks and [credential stuffing](#).
- The [RockYou](#) leak exposed ~32M plaintext passwords, becoming a classic cracking wordlist.
- Modern breach corpora are enormous: [Have I Been Pwned](#) tracks ~17.55B pwned accounts and offers a breached-password checking [API](#).

Top 20 most common passwords according to NordPass

Rank	Password	Count of password uses
1	123456	21,627,656
2	admin	21,030,012
3	12345678	8,274,408
4	123456789	5,673,712
5	12345	3,950,777
6	password	3,545,119
7	Aa123456	2,520,728
8	1234567890	1,418,939
9	Pass@123	1,210,039
10	1qaz!@WSX	1,007,045

How Do I Check Password Strength for Users?

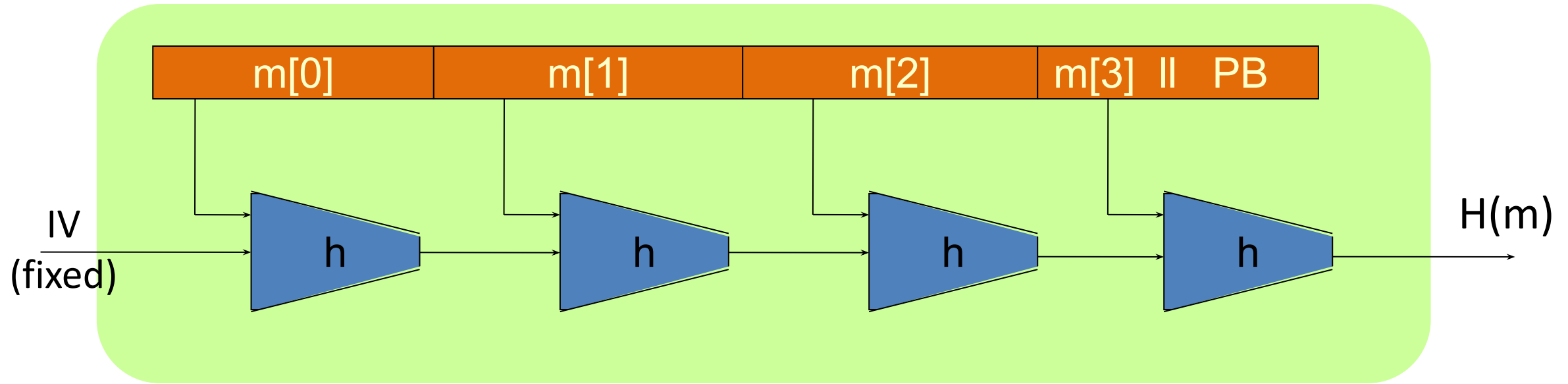
- **Length matters:** prefer long passphrases, e.g., 4–6 random words, rather than short “complex” password (password length is standard)
- **Use a realistic strength estimator:** tools like Dropbox’s [zxcvbn](#) detect patterns, names, dates, keyboard walks, and common substitutions
- **Best practical rule:** use a password manager to generate unique random passwords for every site
- No generally accepted solution today (open problem!)





Back to Merkle-Damgard

The Merkle-Damgård iterated construction



Thm: h collision resistant $\Rightarrow H$ collision resistant

Can we use $H(.)$ to directly build a MAC?

MAC from a Merkle-Damgard Hash Function

$H: X^{\leq L} \rightarrow T$ a C.R. Merkle-Damgard Hash Function

Attempt #1: $S(k, m) = H(k \parallel m)$

This MAC is insecure because:

- Given $H(k \parallel m)$ can compute $H(w \parallel k \parallel m \parallel PB)$ for any w .
- Given $H(k \parallel m)$ can compute $H(k \parallel m \parallel w)$ for any w .
- Given $H(k \parallel m)$ can compute $H(k \parallel m \parallel PB \parallel w)$ for any w .
- Anyone can compute $H(k \parallel m)$ for any m .

Standardized method: HMAC (Hash-MAC)

Most widely used MAC on the Internet.

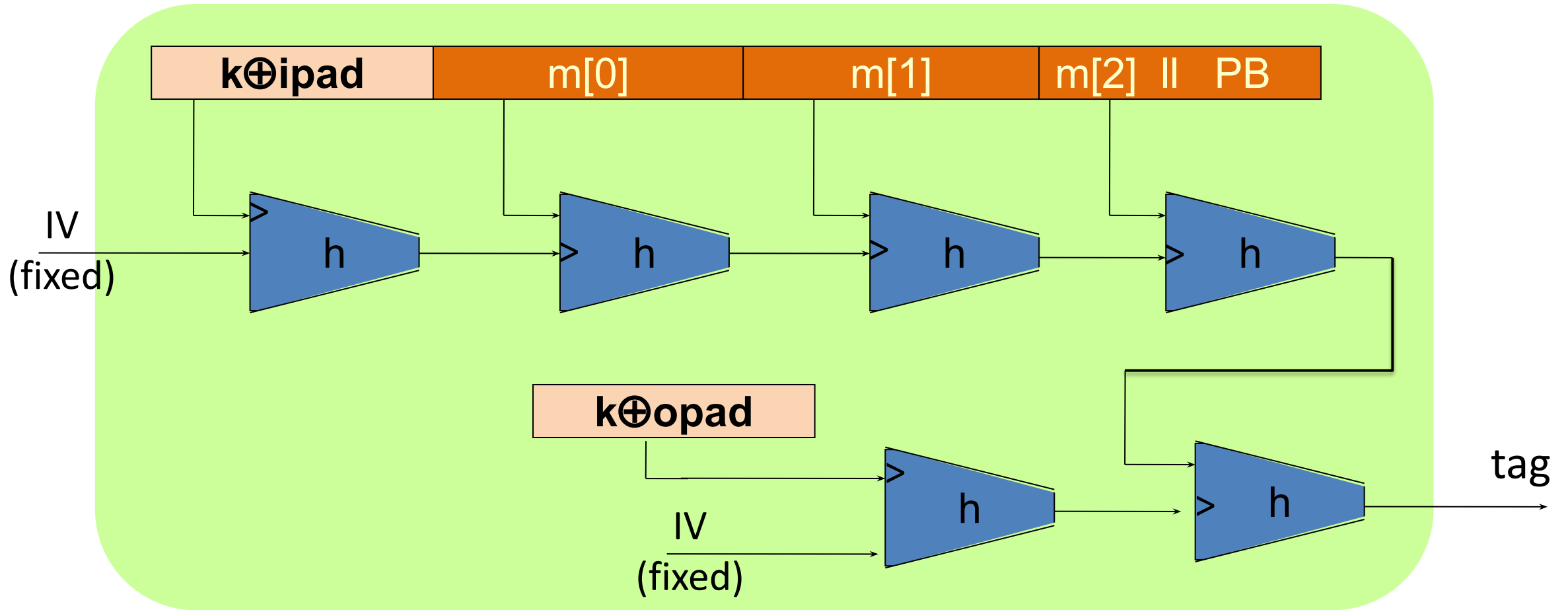
H: hash function.

example: SHA-256 ; output is 256 bits

Building a MAC out of a hash function:

$$\text{HMAC: } S(k, m) = H(k \oplus \text{opad} \parallel H(k \oplus \text{ipad} \parallel m))$$

HMAC in pictures



Similar to the NMAC PRF.

main difference: the two keys k_1, k_2 are dependent

HMAC properties

Built from a black-box implementation of SHA-256.

HMAC is assumed to be a secure PRF

- Can be proven under certain PRF assumptions about $h(.,.)$
- Security bounds similar to NMAC
 - Need $q^2/|T|$ to be negligible ($q \ll |T|^{1/2}$)

In TLS: must support HMAC-SHA1-96

Warning: verification timing attacks [L'09]

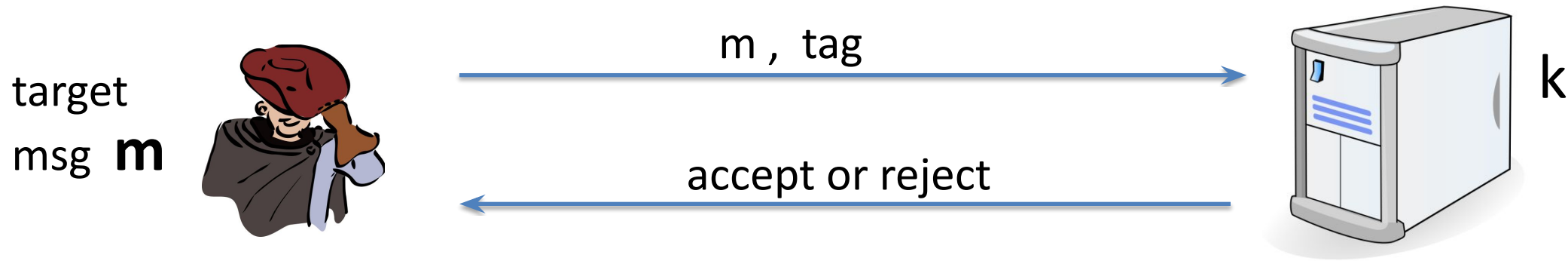
Example: Keyczar crypto library (Python) [simplified]

```
def Verify(key, msg, sig_bytes):  
    return HMAC(key, msg) == sig_bytes
```

The problem: '==' implemented as a byte-by-byte comparison

- Comparator returns false when first inequality found

Warning: verification timing attacks [L'09]



Timing attack: to compute tag for target message m do:

Step 1: Query server with random tag

Step 2: Loop over all possible first bytes and query server.

stop when verification takes a little longer than in step 1

Step 3: repeat for all tag bytes until valid tag found



Defense #1

Make string comparator always take same time (Python) :

```
return false if sig_bytes has wrong length
```

```
result = 0
```

```
for x, y in zip( HMAC(key,msg) , sig_bytes):
```

```
    result |= ord(x) ^ ord(y)
```

```
return result == 0
```

Can be difficult to ensure due to optimizing compiler.

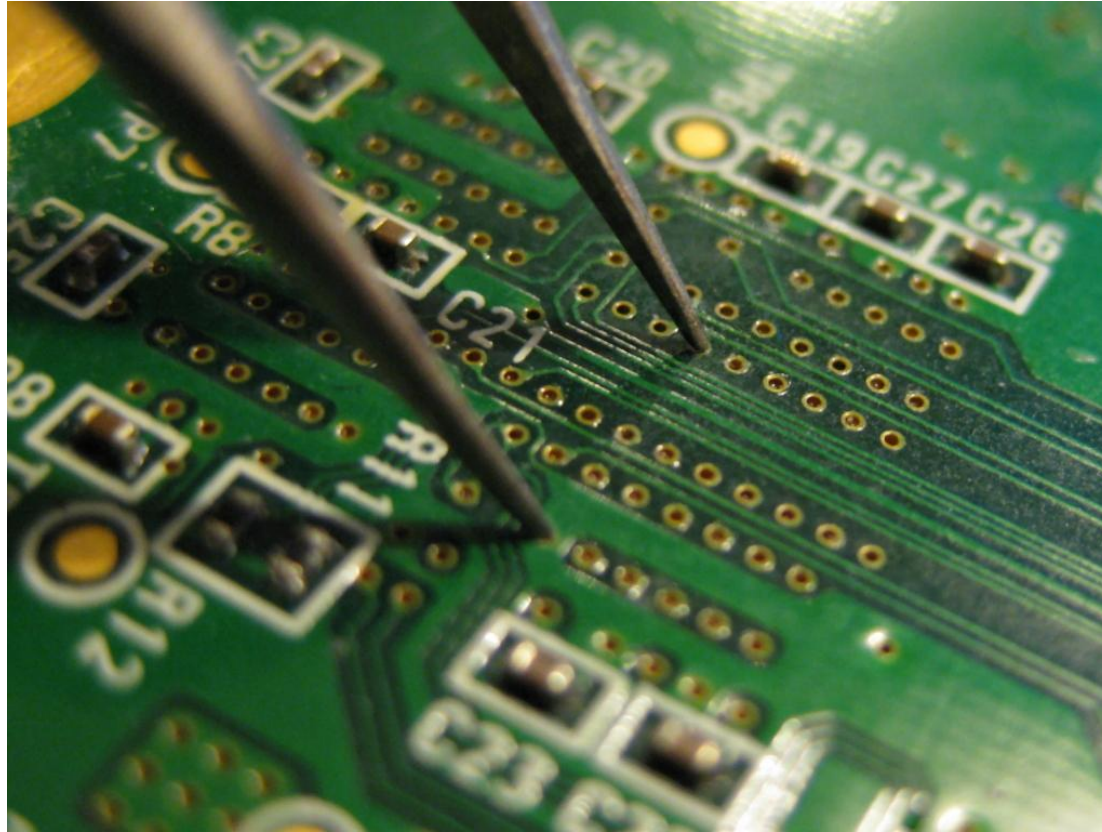
Defense #2

Make string comparator always take same time (Python) :

```
def Verify(key, msg, sig_bytes):  
    mac = HMAC(key, msg)  
    return HMAC(key, mac) == HMAC(key, sig_bytes)
```

Attacker doesn't know values being compared

Hacking Wii - Twilight Hack :)



```
int accept(SHA1_calc) {  
    return (strncmp(SHA1_sig, SHA1_calc, 20) == 0)  
}  
  
// surely that's safe?
```

<https://www.youtube.com/watch?v=4BlpONgj74A&list=PLUbw0IXAmW2rMk5g19MrqcyebecSrrSeJ&index=5>



Hash Tricks and Datastructures

Application Uses for Hashes

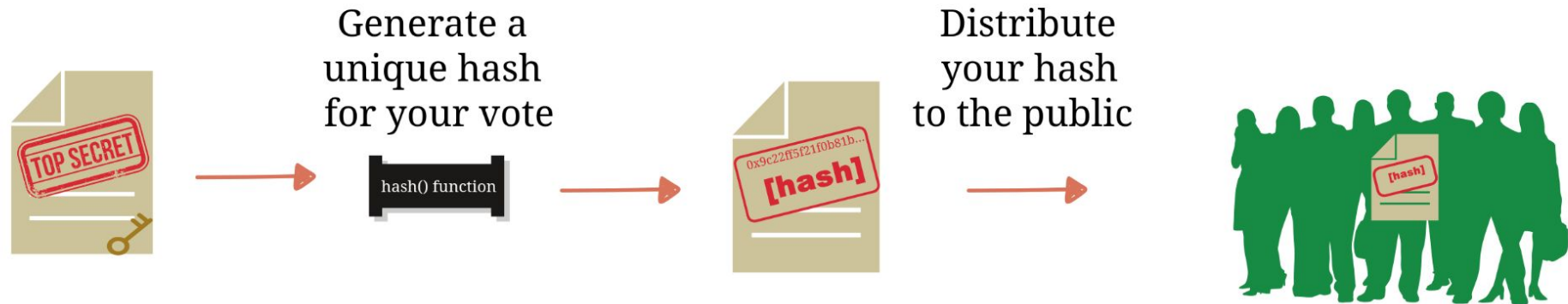
- One-Time Passwords (OTP)
- Data Integrity (tamper-proof)
- Blockchain / micropayments
- Commitment protocols
- and many more

Interview Question #1

You want to prove to another party that you completed an action (e.g., you voted) but not show them your action until after an amount of time has passed (e.g., after election results). How would you implement such a scheme?

Commitment Scheme

1. Commit



Wait for all votes to be committed....

2. Reveal



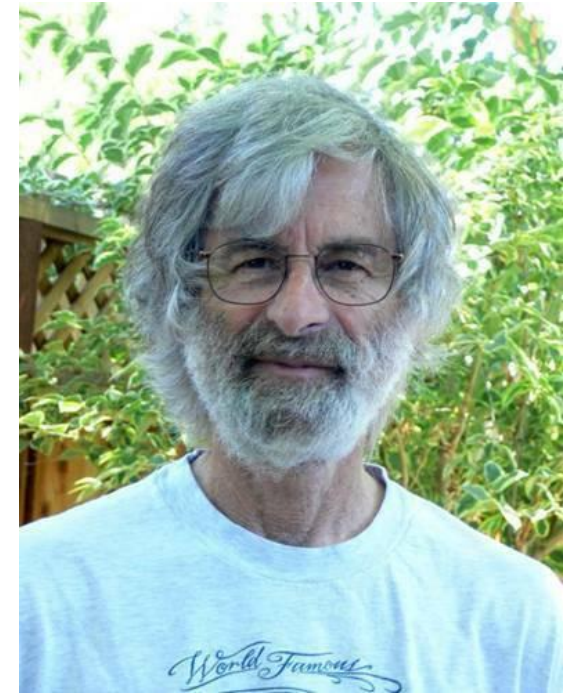
🎉 Count votes and declare winner! 🎉

Interview Question #2

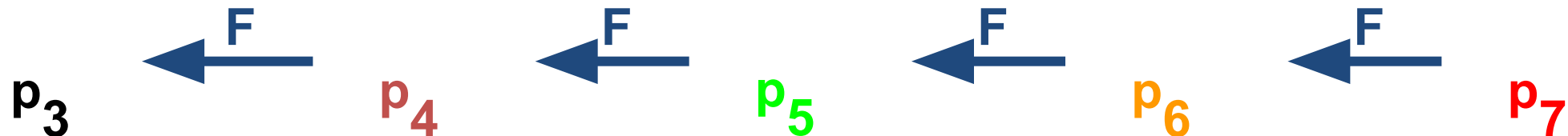
Two people want to simulate a fair die roll (1–6) over the internet, but they don't trust each other. How can they do it in such a way that neither party can cheat?

One-Way Chain Application (Lists)

- One-time password system
- Goal
 - Use a different password at every login
 - Server cannot derive password for next login
- Solution: one-way chain
 - Pick random password P_L
 - Prepare sequence of passwords $P_i = F(P_{i+1})$
 - Use passwords $P_0, P_1, \dots, P_{L-1}, P_L$
 - Server can easily authenticate user

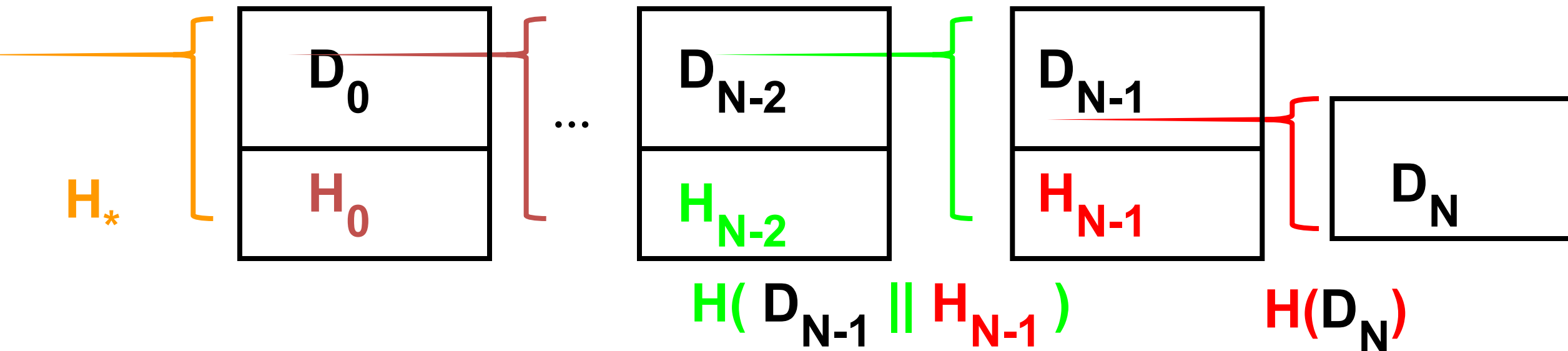


Lamport (1981)



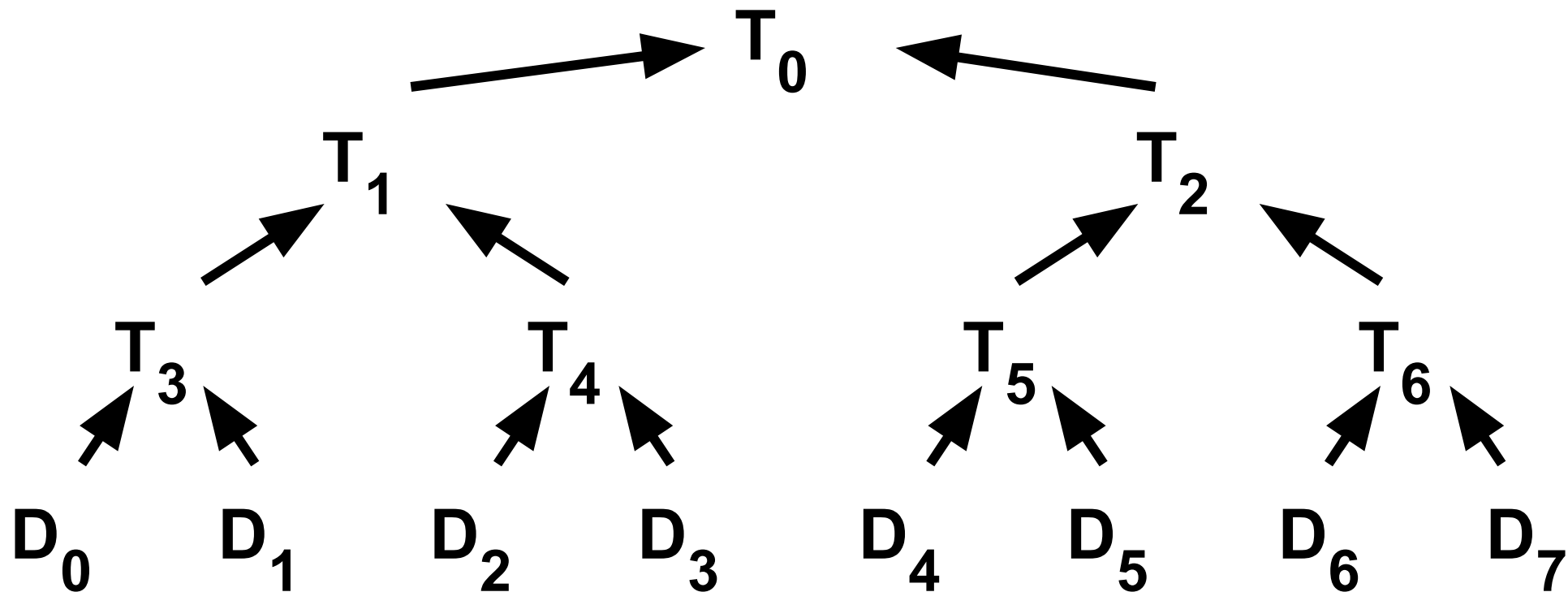
Chained Hashes

- More general construction than one-way hash chains
- Useful for authenticating a sequence of data values $D_0, D_1, \dots, D_N$
- H_* authenticates entire chain



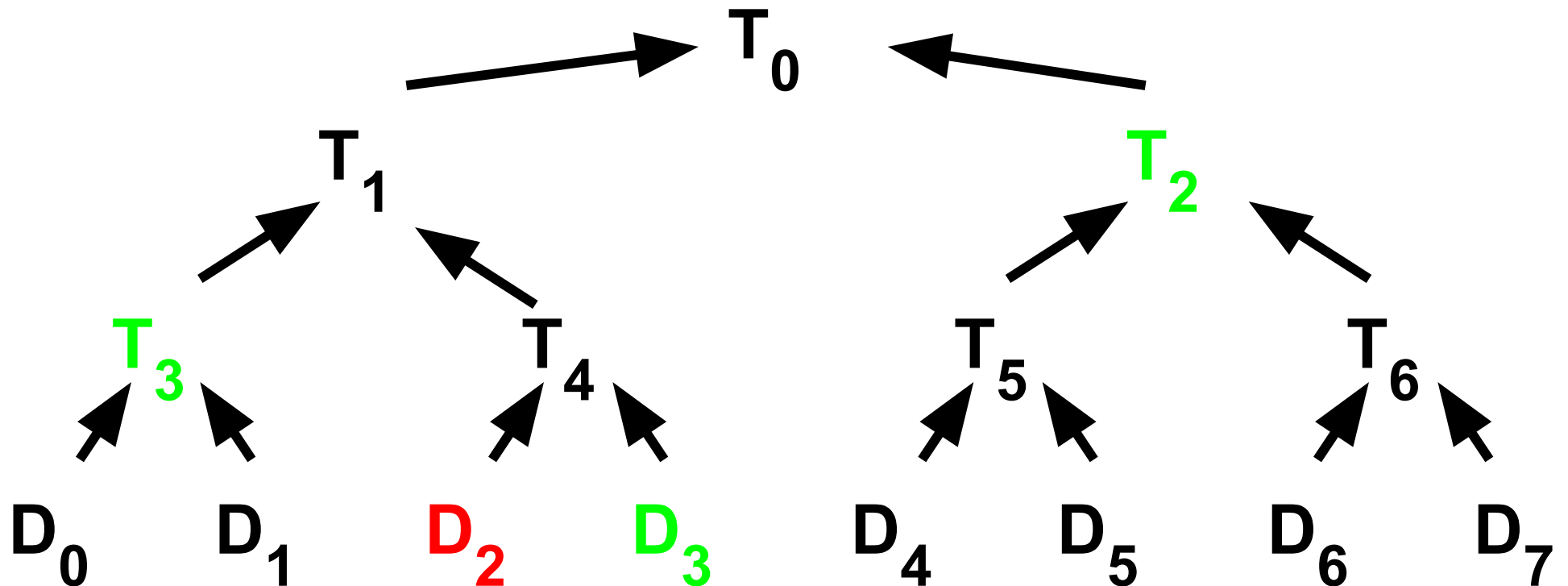
Merkle Hash Trees

- Authenticate a sequence of data values $D_0, D_1, \dots, D_N$
- Construct binary tree over data values



Merkle Hash Trees II

- Verifier knows T_0
- How can verifier authenticate leaf D_i ?
- Solution: recompute T_0 using D_i
- Example authenticate D_2 , send $D_3 T_3 T_2$
- Verify $T_0 = H(H(T_3 \parallel H(D_2 \parallel D_3)) \parallel T_2)$





Authenticated Encryption

Recap: the story so far

Confidentiality: semantic security against a CPA attack

- Encryption secure against **eavesdropping only**

Integrity:

- Existential unforgeability under a chosen message attack
- CBC-MAC, HMAC, *MAC

Can we combine them: encryption secure against **tampering**

- Ensuring both confidentiality and integrity

... but first, some history

Authenticated Encryption (AE): introduced in 2000 [KY'00, BN'00]

Crypto APIs before then: (e.g. MS-CAPI)

- Provide API for CPA-secure encryption (e.g. CBC with rand. IV)
- Provide API for MAC (e.g. HMAC)

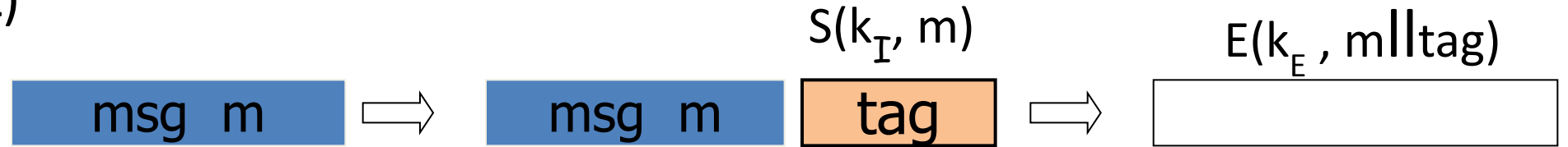
Every project had to combine the two itself without a well defined goal

- Not all combinations provide AE ...

Combining MAC and ENC (CCA)

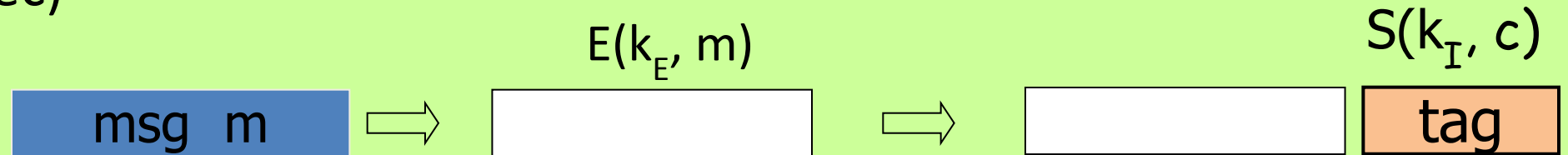
Encryption key k_E . MAC key = k_I

Option 1: (SSL)

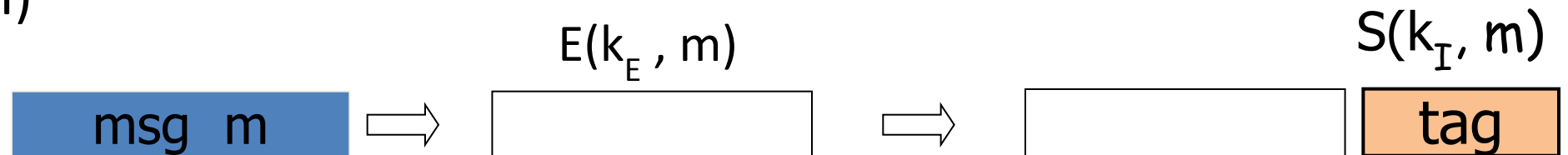


Option 2: (IPsec)

**always
correct**



Option 3: (SSH)



A.E. Theorems

Let (E,D) be CPA secure cipher and (S,V) secure MAC. Then:

1. **Encrypt-then-MAC:** always provides A.E.

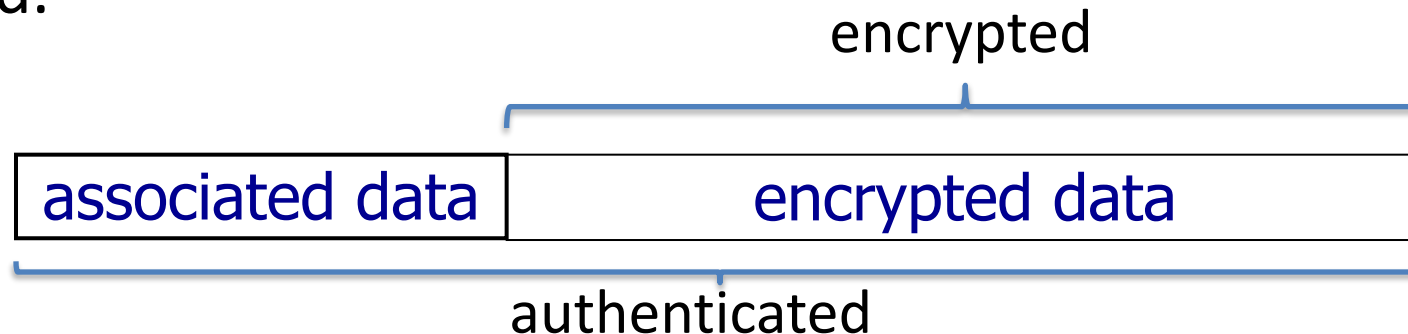
1. **MAC-then-encrypt:** may be insecure against CCA attacks

however: when (E,D) is rand-CTR mode or rand-CBC
M-then-E provides A.E.

Standards (at a high level)

- **GCM:** CTR mode encryption then CW-MAC
(accelerated via Intel's PCLMULQDQ instruction)
- **CCM:** CBC-MAC then CTR mode encryption (802.11i)
- **EAX:** CTR mode encryption then CMAC

All support AEAD: (auth. enc. with associated data). All are nonce-based.

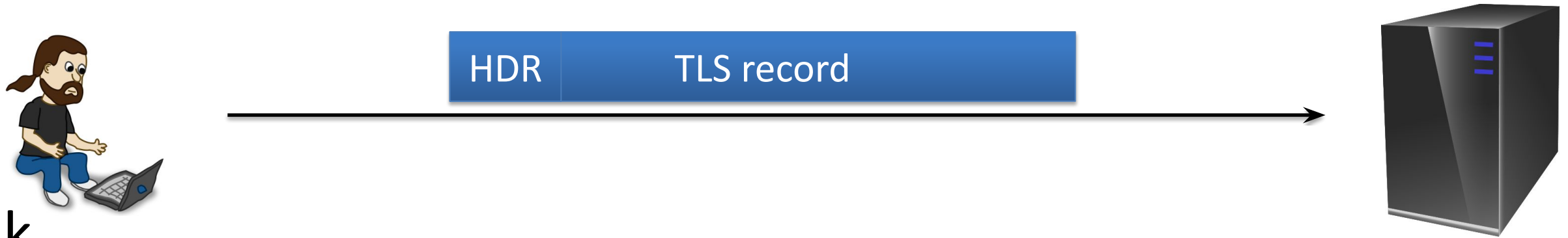


An example API (OpenSSL)

```
int AES_GCM_Init(AES_GCM_CTX *ain,  
    unsigned char *nonce, unsigned long noncelen,  
    unsigned char *key, unsigned int klen )
```

```
int AES_GCM_EncryptUpdate(AES_GCM_CTX *a,  
    unsigned char *aad, unsigned long aadlen,  
    unsigned char *data, unsigned long datalen,  
    unsigned char *out, unsigned long *outlen)
```

The TLS Record Protocol (TLS 1.2)



$k_{b \rightarrow s'}$

$k_{s \rightarrow b}$

Unidirectional keys:

$k_{b \rightarrow s}$

and

$k_{s \rightarrow b}$

$k_{b \rightarrow s'}$

$k_{s \rightarrow b}$

Stateful encryption:

- Each side maintains two 64-bit counters: $ctr_{b \rightarrow s}$, $ctr_{s \rightarrow b}$
- Init. to 0 when session started. $ctr++$ for every record.
- Purpose: replay defense

Ευχαριστώ και καλή μέρα εύχομαι!

Keep hacking!